

DATE : 17.09.2026
DT/NT : DT
LESSON : GIT & GITHUB
SUBJECT: Git & Github-3

BATCH : B 424-020DE

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59



What are we doing today?

Advanced Git

Stash • Reset • Revert
Rebase • Cherry-pick

Versions & Documentation

Tags • Semantic Versioning
GitHub Releases • README

DevOps Automation

CI/CD • GitHub Actions
Secrets • AWS OIDC

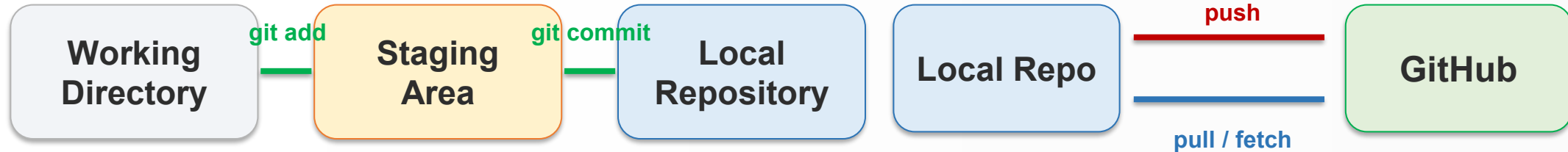
Today we connect Git/GitHub to the DevOps automation workflow.

Day 1: Local Git → Day 2: Collaboration → Day 3: Automation

Day 1–2 Quick Review

LOCAL GIT

REMOTE + TEAM

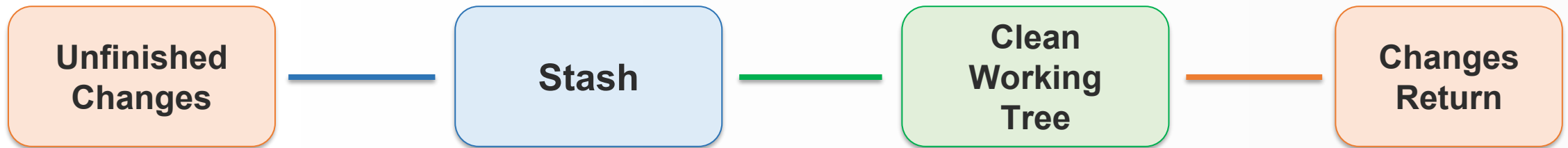


Feature Branch → Pull Request → Code Review → Merge

If these ideas are clear, you are ready for Day 3.

Git Stash — Temporarily Save Unfinished Work

Scenario: Your work is not ready to commit, but you need to switch branches.



Basic commands

```
git stash  
git stash list  
git stash pop
```

git stash

Temporarily stores uncommitted changes.

git stash pop

Restores the latest stash and removes it from the stash list.

Undoing Changes — Choose the Right Tool

Working Directory

Discard an uncommitted file
`git restore <file>`

Staging Area

Remove a file from staging
`git restore --staged <file>`

Local Commit

Move local history /
undo the last local
commit
`git reset ...`

Shared Commit

Create a new commit
that reverses an old commit
`git revert <commit>`

Ask first: “Where is the mistake right now?”

restore → files | reset → local history | revert → safe shared-history undo

git reset — Move Local History

Use reset carefully. It is mainly a local-history tool.

--soft

```
git reset --soft HEAD~1
```

Commit is removed.
Changes stay STAGED.

--mixed (default)

```
git reset --mixed HEAD~1
```

Commit is removed.
Changes stay in the WORKING DIRECT

--hard

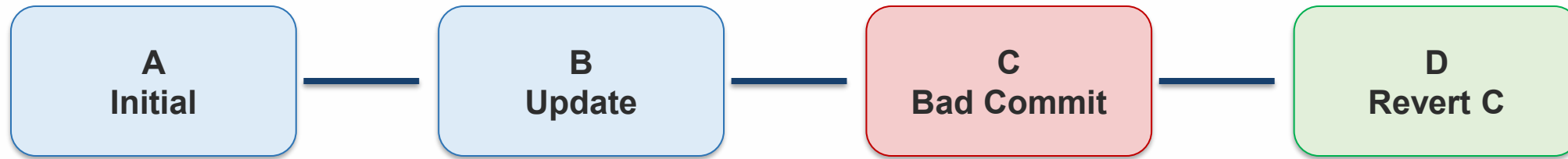
```
git reset --hard HEAD~1
```

Commit and local changes are discarded.

⚠ --hard can destroy uncommitted work. Do not use it casually.

git revert — Safely Undo a Shared Commit

Revert does not delete the old commit. It creates a new commit that reverses it.



Example

```
git log --oneline  
git revert <commit-id>
```

Best for pushed/shared history

The project history remains visible and auditable.

Git creates a new reversing commit.

Reset vs Revert

Both can undo a commit — but they change history differently.

git reset

- Moves the branch pointer / rewrites local history
- Good for local, unshared cleanup
- **Can discard work with --hard**
- Avoid rewriting commits teammates already use

git revert

- Creates a new reversing commit
- Preserves the existing history
- **Safer for shared / pushed commits**
- Easy to audit later

Simple rule: Local/private history → reset may be appropriate | Shared history → prefer revert

Merge vs Rebase

Two ways to integrate branch changes.

MERGE

```
      A---B---C  main
     /
    D---E  feature
     \
      M
```

Merge preserves the branch structure and creates a merge commit when needed.

REBASE

```
A---B---C---D'---E'
```

feature commits are replayed
on top of main

Rebase can create a cleaner linear history, but it rewrites commit IDs.

⚠ Avoid rebasing shared commits that other developers are already using.

git cherry-pick — Take One Specific Commit

Use cherry-pick when you need one commit, not an entire branch.

main

A---B---C

feature

A---B---D---E

↑

Need D

git cherry-pick <commit-id>

After cherry-pick

A---B---C---D'

D is copied as a new commit on the current branch.

Git Tags — Mark Important Versions

A tag gives a meaningful name to a specific commit.

Create and push an annotated tag

```
A --- B --- C --- D
      ↑
    v1.0.0
```

```
git tag
git tag -a v1.0.0 -m "Version 1.0.0"
git push origin v1.0.0
```

Commit

Moving project history

Branch

Moving line of development

Tag

Fixed label on one commit

Semantic Versioning — MAJOR.MINOR.PATCH

v1.2.3

MAJOR

Breaking / incompatible change

MINOR

New backward-compatible feature

PATCH

Backward-compatible bug fix

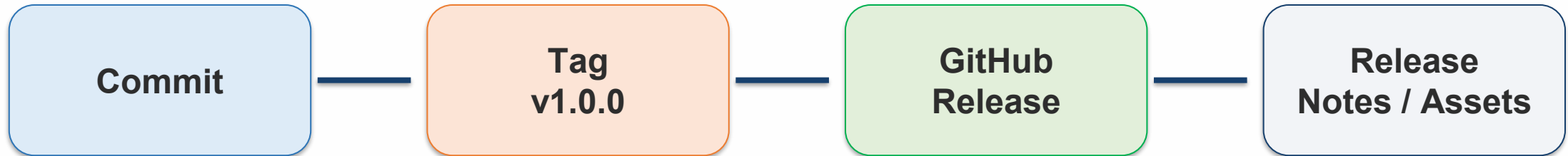
v1.0.0 First stable release

v1.1.0 New feature

v1.1.1 Bug fix

v2.0.0 Breaking change

GitHub Releases — Publish a Version



• Typical release information

- Version / tag
- What changed
- Known issues
- Optional downloadable artifacts

DevOps connection

A release/tag can trigger a CI/CD workflow.
Example: build an image, publish an artifact, or deploy.

Writing a Good Project README

- **A useful README should answer:**

- What is this project?
- How do I install it?
- How do I run it?
- Which technologies are used?
- How is it deployed?

Example structure

```
# Node.js Application

## Description
Simple app deployed on AWS.

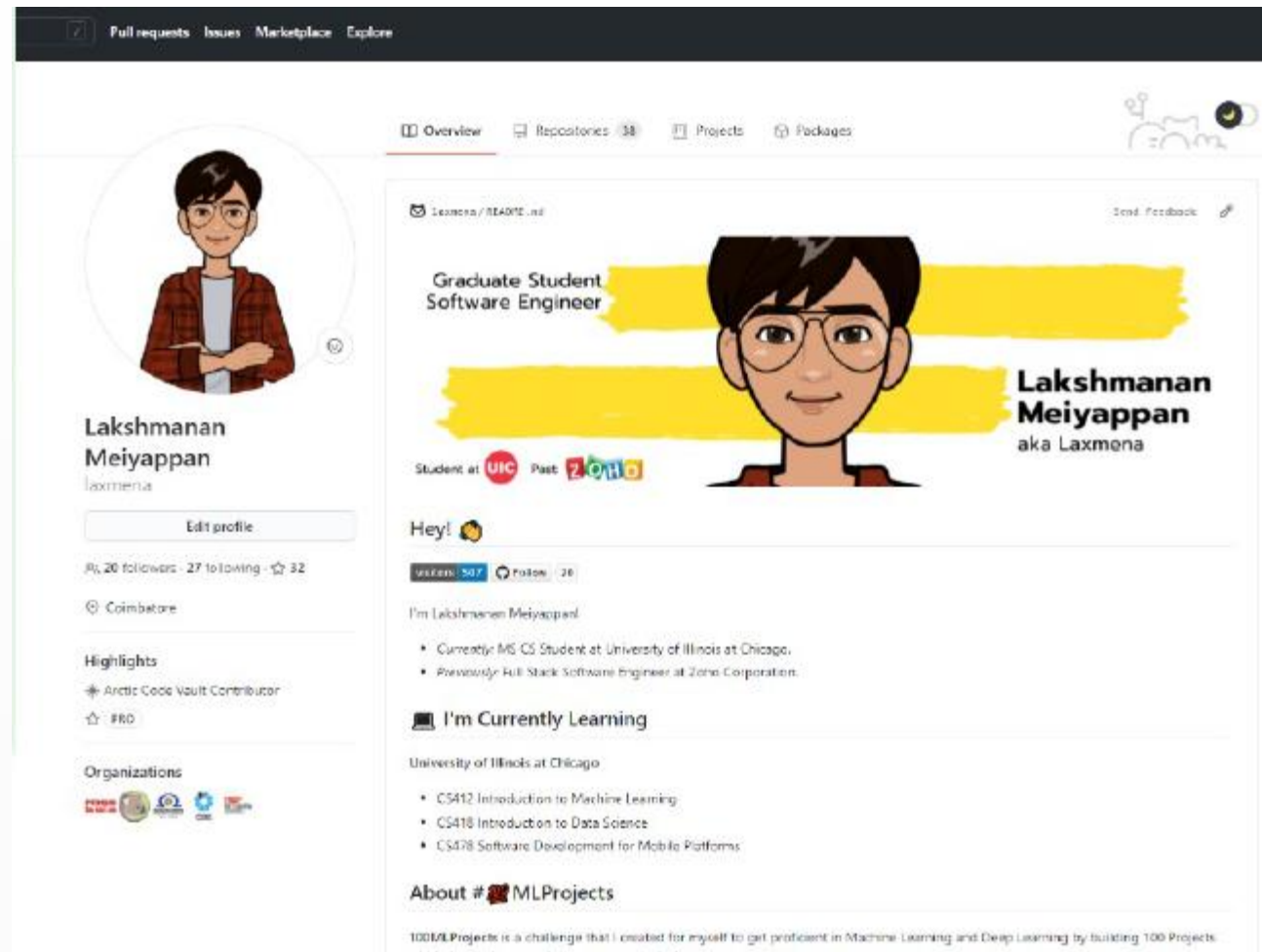
## Prerequisites
- Node.js
- Docker
- AWS CLI

## Run
npm start

## Deployment
GitHub Actions → AWS
```

GitHub Readme.md

Readme Generator

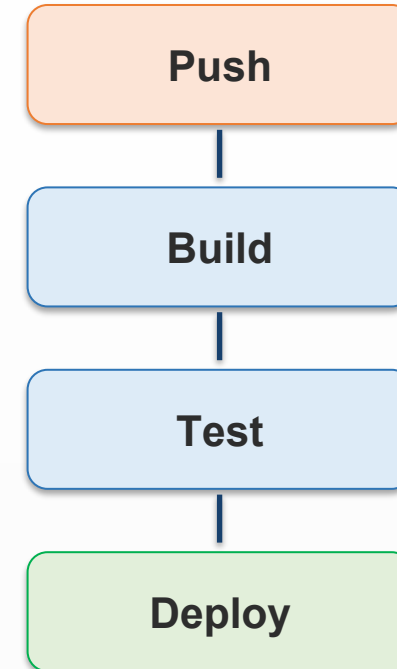


From git push to CI/CD

Without automation



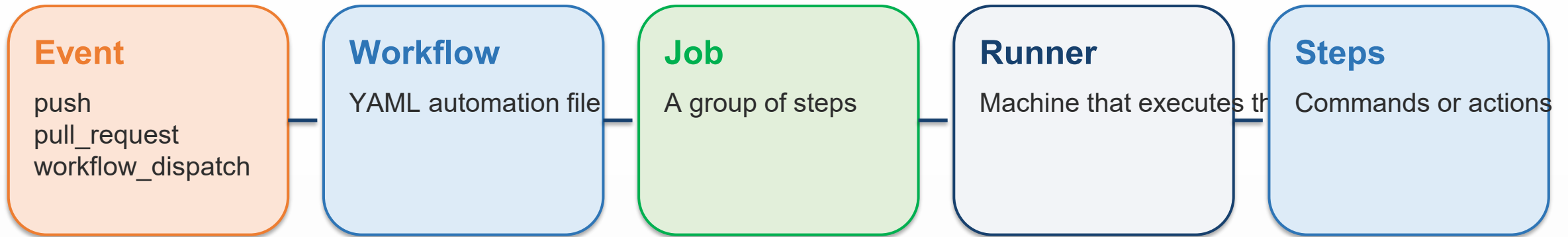
With CI/CD automation



CI = Continuous Integration | CD = Continuous Delivery / Deployment

What is GitHub Actions?

GitHub Actions runs automation when events happen in your repository.



A Git event can automatically start build, test, security, packaging, or deployment work.

GitHub Actions Building Blocks

EVENT

What starts the workflow?
Examples: push, PR, manual

WORKFLOW

The YAML automation definition

JOB

A collection of related steps

RUNNER

The machine that executes a job

STEP

One command or reusable action

ACTION

Reusable packaged automation

Where Do GitHub Actions Workflows Live?

GitHub automatically looks for workflow YAML files under:

```
project/
├── app.js
├── README.md
└── .github/
    └── workflows/
        ├── ci.yml
        └── deploy.yml
```

.github/workflows/

Each .yml / .yaml file is a workflow definition.

Commit the workflow file just like normal project code.

Workflow changes are version-controlled, reviewed, and shared with the repository.

First GitHub Actions Workflow

.github/workflows/first.yml

```
name: First Workflow

on:
  push:
    branches: [main]

jobs:
  demo:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - run: echo "GitHub Actions is running!"
      - run: ls -la
```

name

Workflow display name

on

Trigger event

jobs

Work to perform

runs-on

Runner OS

steps

Actions and shell commands

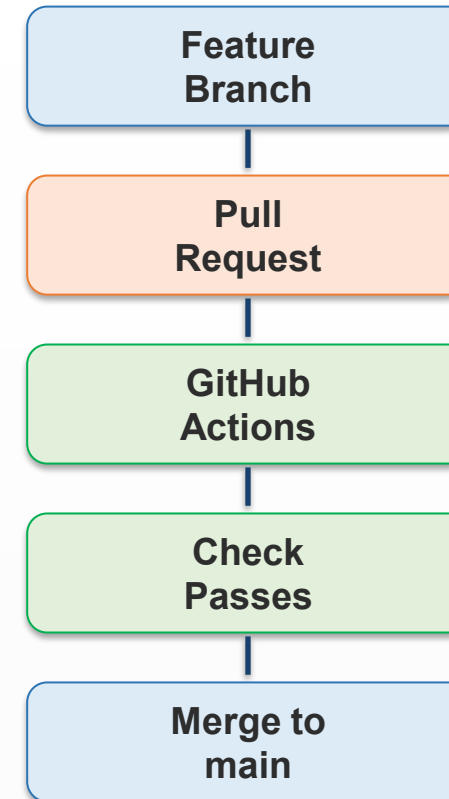
Pull Request Validation Workflow

[.github/workflows/pr-check.yml](#)

```
name: PR Check

on:
  pull_request:
    branches: [main]

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - run: echo "PR validation passed"
```



This connects Day 2 Pull Requests with Day 3 automation.

GitHub Secrets and Variables

Never place real credentials directly inside a workflow file.

DO NOT

AWS_ACCESS_KEY_ID: AKIA...
PASSWORD: mypassword
TOKEN: abc123

USE

Repository / Environment
Secrets and Variables

Reference them at runtime.

Workflow references

```
${{  
  secrets.DEMO_MESSAGE  
}}  
${{ vars.APP_ENV }}
```

Secrets

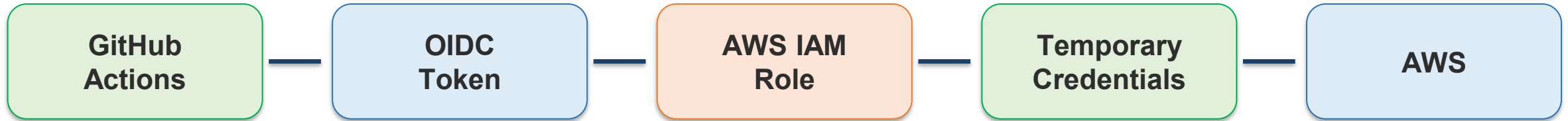
Sensitive values. Masked in logs when supported.

Variables

Non-sensitive configuration values.

GitHub Actions → AWS with OIDC

Recommended concept: use short-lived AWS credentials instead of storing long-lived access keys.



Conceptual workflow snippet

```
permissions:
  id-token: write
  contents: read

- uses: aws-actions/configure-aws-credentials@v6
  with:
    role-to-assume: arn:aws:iam::<id>:role/GitHubActionsRole
    aws-region: us-east-1
```

Why OIDC?

No long-lived AWS keys stored in GitHub.
AWS issues temporary credentials.
IAM trust policy controls who can assume the role.

Complete DevOps Git Workflow



The Git workflow is now part of the delivery pipeline.

Git

tracks changes

GitHub

coordinates collaboration

GitHub Actions

automates validation and delivery

AWS

runs the workload

Day 3 Final Challenge

Complete this workflow with a simple repository.

1. Make an unfinished change → stash it → restore it
2. Create a bad local commit → practice reset
3. Create a pushed bad commit → practice revert
4. Create and push tag v1.0.0
5. Create a GitHub Release for v1.0.0
6. Add a push-triggered GitHub Actions workflow
7. Add a PR validation workflow and verify it runs

Useful commands

```
git stash
git reset --soft HEAD~1
git revert <commit-id>
git tag -a v1.0.0 -m "Version 1.0.0"
git push origin v1.0.0
```

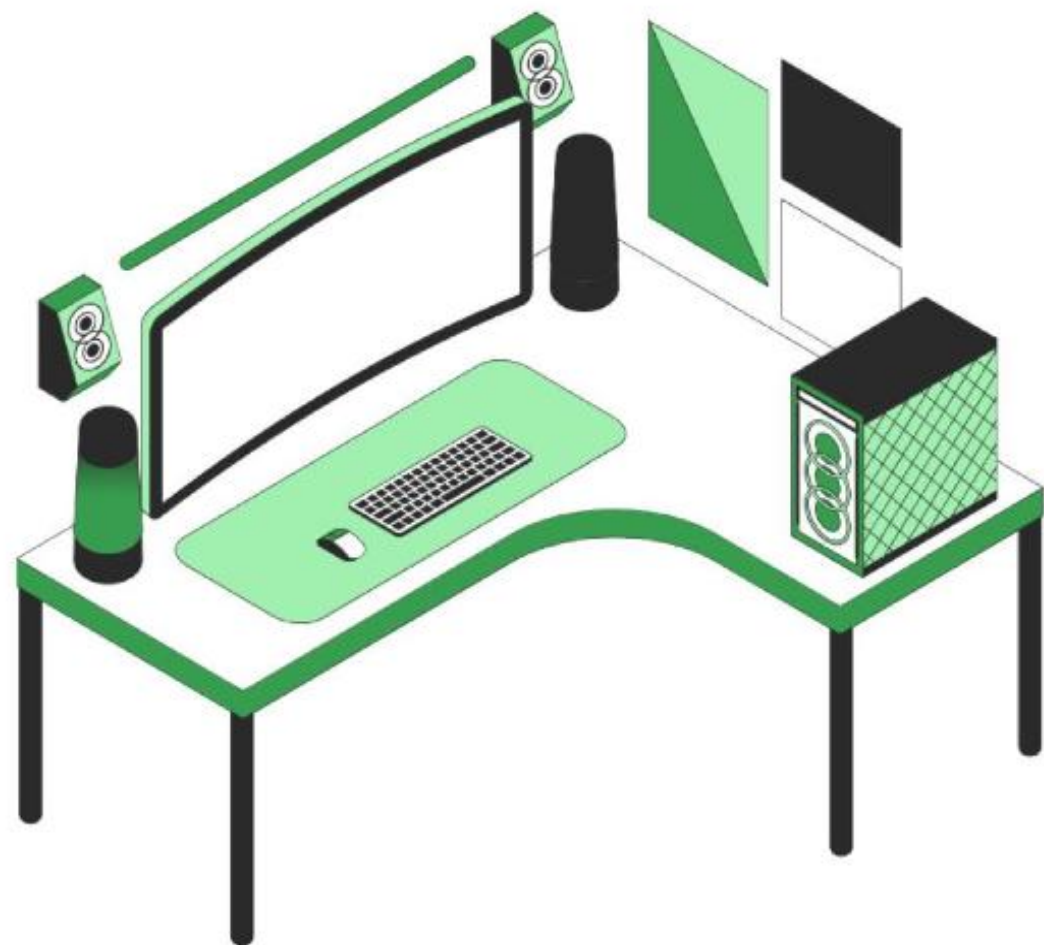
Done condition: You can explain what triggers the workflow and where the result is visible in GitHub.

GitHub Desktop Installation



Installation Link
<https://desktop.github.com/>





Do you have any questions?

Send it to us! We hope you learned something new.